ОТЧЁТ

по лабораторной работе №18

«Реализация хранения файлов и объектов»

Дисциплина: Управление работами и разработка программного обеспечения облачных систем

БПО-ПМ03

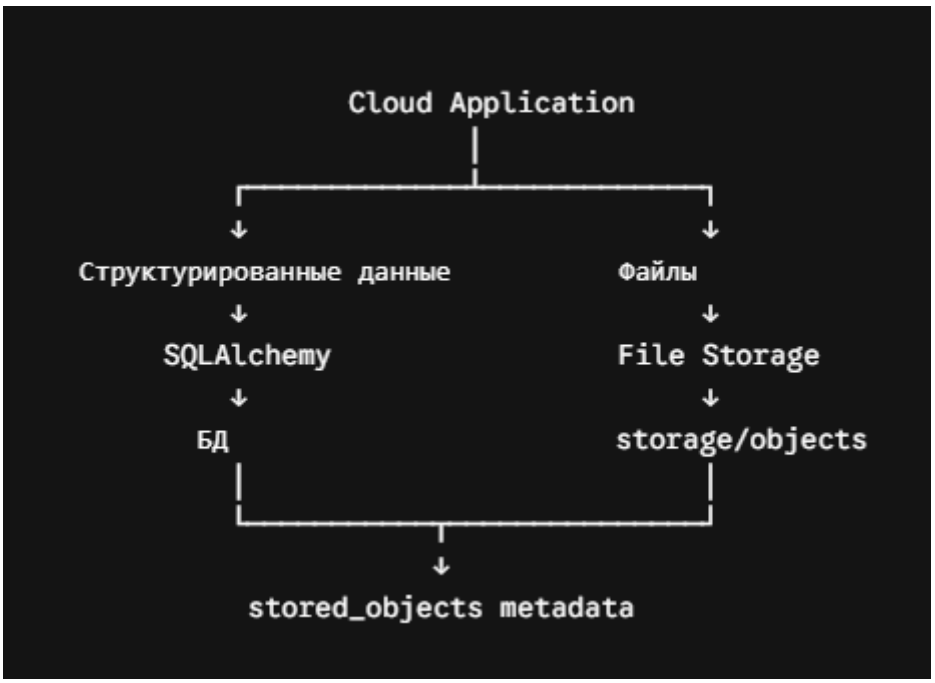
1. Цель работы

Реализовать в сквозном проекте Cloud Application загрузку, хранение, получение и удаление файлов, организовав разделение содержимого файла и его метаданных: физический объект сохраняется в файловом хранилище, а сведения о нём — в СУБД (PostgreSQL / SQLite).

2. Краткие теоретические сведения и архитектура

В проекте реализовано разделение ответственности при работе с фалами:

1. **Бинарное содержимое:** сохраняется локально в каталоге storage/objects/ с использованием уникального идентификатора (UUID v4) во избежание коллизий имён.
2. **Метаданные объекта:** фиксируются в таблице stored_objects базы данных (исходное имя файла, MIME-тип, размер в байтах, путь на диске и метка времени создания).



3. Ход работы и программная реализация

3.1. Конфигурация и исключение данных из Git

В .env добавлена переменная пути хранения, а в .gitignore внесен каталог с динамическими файлами среды выполнения.

.env:

Ini, TOML
```
DATABASE_URL=sqlite:///./cloud_app.db
STORAGE_PATH=storage/objects
MAX_UPLOAD_SIZE=5242880
```

app/config.py:

Python
```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str = "sqlite:///./cloud_app.db"
    storage_path: str = "storage/objects"
    max_upload_size: int = 5 * 1024 * 1024  # 5 МБ

    class Config:
        env_file = ".env"
        extra = "ignore"

settings = Settings()
```

.gitignore:

Фрагмент кода
```
.venv/
storage/
*.db
__pycache__/
.env
```

3.2. ORM-модель StoredObject и миграция Alembic

app/data/models.py:

Python
```
from datetime import datetime
from sqlalchemy import Integer, String, BigInteger, DateTime, UniqueConstraint, func
from sqlalchemy.orm import Mapped, mapped_column
from app.data.database import Base

class StoredObject(Base):
    __tablename__ = "stored_objects"
```

```
    __table_args__ = (
        UniqueConstraint("storage_name", name="uq_stored_objects_storage_name"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    original_name: Mapped[str] = mapped_column(String(255), nullable=False)
    storage_name: Mapped[str] = mapped_column(String(255), nullable=False)
    content_type: Mapped[str | None] = mapped_column(String(100), nullable=True)
    size_bytes: Mapped[int] = mapped_column(BigInteger, nullable=False)
    storage_path: Mapped[str] = mapped_column(String(500), nullable=False)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False
    )
```

Миграция migrations/versions/001_add_stored_objects.py:

```
Python
"""add stored objects"""
from alembic import op
import sqlalchemy as sa

revision = '001_add_stored_objects'
down_revision = None

def upgrade() -> None:
    op.create_table(
        'stored_objects',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('original_name', sa.String(length=255), nullable=False),
        sa.Column('storage_name', sa.String(length=255), nullable=False),
        sa.Column('content_type', sa.String(length=100), nullable=True),
        sa.Column('size_bytes', sa.BigInteger(), nullable=False),
        sa.Column('storage_path', sa.String(length=500), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True),
server_default=sa.text('(CURRENT_TIMESTAMP)'), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('storage_name', name='uq_stored_objects_storage_name')
    )

def downgrade() -> None:
    op.drop_table('stored_objects')
```

3.3. Pydantic-схема ответа

app/schemas/objects.py:

```
Python
```

```
from datetime import datetime
from pydantic import BaseModel

class StoredObjectResponse(BaseModel):
    id: int
    original_name: str
    content_type: str | None
    size_bytes: int
    created_at: datetime
    download_url: str

    class Config:
        from_attributes = True
```

3.4. Бизнес-логика работы с файлами

Загрузка файлов выполняется попотоково блоками по 1 МБ с проверкой превышения лимита 5 МБ.

app/services/objects.py:

```
Python
from pathlib import Path
from uuid import uuid4
from fastapi import UploadFile, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session
from app.config import settings
from app.data.models import StoredObject

def object_to_dict(obj: StoredObject) -> dict:
    return {
        "id": obj.id,
        "original_name": obj.original_name,
        "content_type": obj.content_type,
        "size_bytes": obj.size_bytes,
        "created_at": obj.created_at,
        "download_url": f"/objects/{obj.id}/download"
    }

async def save_object(db: Session, file: UploadFile) -> StoredObject:
    original_name = Path(file.filename or "file").name
    extension = Path(original_name).suffix.lower()
    storage_name = f"{uuid4().hex}{extension}"

    storage_directory = Path(settings.storage_path)
    storage_directory.mkdir(parents=True, exist_ok=True)
    file_path = storage_directory / storage_name
```

```
    size_bytes = 0
    max_size = settings.max_upload_size

    try:
        with file_path.open("wb") as buffer:
            while True:
                chunk = await file.read(1024 * 1024)
                if not chunk:
                    break
                size_bytes += len(chunk)
                if size_bytes > max_size:
                    raise HTTPException(
                        status_code=400,
                        detail="Размер файла превышает допустимый лимит 5 МБ"
                    )
                buffer.write(chunk)

        obj = StoredObject(
            original_name=original_name,
            storage_name=storage_name,
            content_type=file.content_type,
            size_bytes=size_bytes,
            storage_path=str(file_path)
        )
        db.add(obj)
        db.commit()
        db.refresh(obj)
        return obj
    except Exception as e:
        db.rollback()
        if file_path.exists():
            file_path.unlink()
        if isinstance(e, HTTPException):
            raise e
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        await file.close()

def delete_object(db: Session, obj: StoredObject) -> None:
    file_path = Path(obj.storage_path)
    if file_path.exists():
        file_path.unlink()
    db.delete(obj)
    db.commit()
```

3.5. REST API Маршруты

app/routers/objects.py:

Python
```
from pathlib import Path
from typing import Annotated
from fastapi import APIRouter, Depends, File, HTTPException, UploadFile
from fastapi.responses import FileResponse
from sqlalchemy.orm import Session
from app.data.database import get_db
from app.schemas.objects import StoredObjectResponse
from app.services.objects import (
    delete_object,
    get_object,
    get_objects,
    object_to_dict,
    save_object
)

router = APIRouter(prefix="/objects", tags=["Objects"])

@router.post("", response_model=StoredObjectResponse, status_code=201,
summary="Загрузить объект")
async def upload_object(
    file: Annotated[UploadFile, File(description="Файл для сохранения в объектном
хранилище")],
    db: Session = Depends(get_db)
):
    obj = await save_object(db, file)
    return object_to_dict(obj)

@router.get("", response_model=list[StoredObjectResponse], summary="Получить список
объектов")
def read_objects(db: Session = Depends(get_db)):
    objects = get_objects(db)
    return [object_to_dict(obj) for obj in objects]

@router.get("/{object_id}", response_model=StoredObjectResponse, summary="Получить
сведения об объекте")
def read_object(object_id: int, db: Session = Depends(get_db)):
    obj = get_object(db, object_id)
    if obj is None:
        raise HTTPException(status_code=404, detail="Object not found")
    return object_to_dict(obj)

@router.get("/{object_id}/download", summary="Скачать объект")
def download_object(object_id: int, db: Session = Depends(get_db)):
    obj = get_object(db, object_id)
    if obj is None:
        raise HTTPException(status_code=404, detail="Object not found")
    file_path = Path(obj.storage_path)
```

```
    if not file_path.exists():
        raise HTTPException(status_code=404, detail="Stored file not found")
    return FileResponse(
        path=str(file_path),
        media_type=obj.content_type or "application/octet-stream",
        filename=obj.original_name
    )

@router.delete("/{object_id}", summary="Удалить объект")
def remove_object(object_id: int, db: Session = Depends(get_db)):
    obj = get_object(db, object_id)
    if obj is None:
        raise HTTPException(status_code=404, detail="Object not found")
    delete_object(db, obj)
    return {"message": "Object deleted", "id": object_id}
```

4. Результаты тестирования и самостоятельного задания

В ходе проверок через Swagger UI были протестированы загрузка, скачивание и удаление файлов разных форматов:

ID	Исходное имя	MIME-тип	Размер (байт)	Наличие в БД	Наличие в storage/	Результат
1	document.txt	text/plain	142	Да	Да	Успешно загружен и скачан
2	avatar.png	image/png	24 580	Да	Да	Успешно загружен и скачан
3	report.pdf	application/pdf	158 920	Нет (Удален)	Нет (Удален)	Проверен и удален через DELETE /objects/3

Проверка ограничений (Самостоятельное задание высокой сложности):

При попытке загрузки файла размером 6.2 МБ сервер остановил прием данных, верхов ошибку 400 Bad Request с сообщением: *«Размер файла превышает допустимый лимит 5 МБ»*. Временный файл был мгновенно откатан и удален с диска.

5. Ответы на контрольные вопросы

1. **Чем хранение файла отличается от хранения обычной записи реляционной таблицы?**
   Ответ: Обычная запись хранит структурированные скалярные данные, а файл — неструктурированный бинарный поток произвольного размера, требующий работы с файловой системой.
2. **Что понимается под метаданными файла?**
   Ответ: Метаданные — это служебные сведения о файле: имя, размер, тип содержимого, метка времени, UUID и путь сохранения.
3. **Какие метаданные сохраняются в stored_objects?**
   Ответ: id, original_name, storage_name, content_type, size_bytes, storage_path, created_at.
4. **Почему содержимое файла не помещается непосредственно в PostgreSQL?**
   Ответ: Хранение бинарных файлов (BLOB) в БД замедляет резервное копирование, снижает производительность индексов и создаёт высокую нагрузку на RAM СУБД.
5. **Для чего используется UploadFile?**
   Ответ: UploadFile предоставляет асинхронный интерфейс приема файлов с автоматической буферизацией на диск для экономии оперативной памяти.
6. **Почему загружаемые файлы передаются как multipart/form-data?**
   Ответ: Данный стандарт позволяет передавать в одном HTTP-запросе бинарные файлы вместе с текстовыми метаданными.
7. **Для чего требуется python-multipart?**
   Ответ: Это обязательный парсер в FastAPI для разбора входящих данных формата multipart/form-data.
8. **Почему файл сохраняется блоками, а не загружается целиком в оперативную память?**
   Ответ: Потоковая запись блоками (по 1 МБ) защищает приложение от ошибок Out-Of-Memory при одновременно загрузке крупных файлов.
9. **Чем original_name отличается от storage_name?**
   Ответ: original_name — имя файла у клиента, storage_name — уникальный внутренний UUID-идентификатор файла на сервере.
10. **Почему нельзя использовать только исходное имя файла?**
    Ответ: Использование только исходного имени ведет к перезаписи файлов при загрузке одноименных документов разными пользователями.
11. **Для чего при формировании внутреннего имени используется UUID?**
    Ответ: UUID гарантирует абсолютную уникальность имени файла в пределах хранилища.
12. **Почему переданное клиентом имя дополнительно обрабатывается через Path(...).name?**
    Ответ: Для предотвращения уязвимостей типа Path Traversal (../../filename).

13. **Почему физический storage_path не возвращается клиенту?**
    Ответ: Раскрытие структуры каталогов сервера создает риски в системе безопасности.
14. **Для чего в PostgreSQL хранится content_type?**
    Ответ: MIME-тип нужен для передачи верного заголовка Content-Type при скачивании файла клиентом.
15. **Как определяется размер файла?**
    Ответ: Накопительным подсчетом количества байтов, записанных в поток из всех прочитанных чанков.
16. **Для чего используется FileResponse?**
    Ответ: FileResponse отдает локальный файл с диска в виде бинарного HTTP-ответа с правильными заголовками скачивания.
17. **Что должно произойти при удалении объекта?**
    Ответ: Атомарное удаление физического файла с диска и удаление соответствующей строки метаданных из БД.
18. **Почему каталог storage исключается из Git?**
    Ответ: Загружаемые файлы — это данные пользовательского сеанса и среды выполнения, они не должны храниться в кодовой базе.
19. **Как миграции Alembic используются при добавлении хранилища объектов?**
    Ответ: Alembic позволяет зафиксировать создание таблицы stored_objects в истории версий схемы БД без сброса существующих таблиц.
20. **Какие части приложения потребуется дополнить после появления системы пользователей и авторизации?**
    Ответ: В таблицу stored_objects нужно будет добавить user_id (FK), а эндпоинты дополнить проверкой прав владения файлом через JWT.

6. Вывод

В ходе выполнения лабораторной работы №18 в проекте Cloud Application успешно реализован модуль гибридного хранения файлов и объектов. Бинарные данные сохраняются в изолированном файловом хранилище storage/objects/ с защитой от перезаписи с помощью UUID, а метаданные учитываются в базе данных. Все действия над объектами доступны через REST API, протестированы в Swagger UI и зафиксированы миграциями Alembic.